

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors.

Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I _____ (Name / Reg No)
 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

1. Signed Integer Addition using 2's Complement

In 8-bit signed representation, the range is -128 to $+127$.

Given:

$$+120 = 01111000$$

$$+50 = 00110010$$

Addition:

$$01111000 (+120)$$

$$+ 00110010 (+50)$$

$$10101010$$

The result 10101010 is interpreted as a negative number in 2's complement, although two positive numbers were added. This indicates overflow.

Why overflow occurs

The actual mathematical result is:

$$120 + 50 = 170$$

But $+170$ cannot be represented using 8-bit signed integers because the maximum is $+127$.

Debugging and Solution

Overflow occurs when:

Two positive numbers produce a negative result, or

Two negative numbers produce a positive result.

For signed addition:

$$\text{Overflow} = \text{Carry into MSB} \text{ XOR } \text{Carry out of MSB}$$

The processor should check the overflow flag after addition.

2. Ripple Carry Adder and Carry Look-Ahead Adder

A Ripple Carry Adder (RCA) consists of several full adders connected in sequence. The carry generated by one bit must propagate to the next bit.

For example:

$$A_0 B_0 \rightarrow \text{FA} \rightarrow C_1$$

↓

$$A_1 B_1 \rightarrow \text{FA} \rightarrow C_2$$

↓

$$A_2 B_2 \rightarrow \text{FA} \rightarrow C_3$$

↓

...

The main problem is carry propagation delay. The final result cannot be obtained until the carry passes through all stages.

Debugging

For each bit:

$$P_i = A_i \text{ XOR } B_i$$

$$G_i = A_i \text{ AND } B_i$$

where:

P_i = Propagate

G_i = Generate

The carry is:

$$C_{i+1} = G_i + P_i C_i$$

A ripple carry adder calculates these carries sequentially.

Optimized Solution: Carry Look-Ahead Adder

A Carry Look-Ahead Adder (CLA) calculates carries in parallel using propagate and generate signals.

For example:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

Thus, the carry does not have to wait for every previous full adder.

Advantages

Reduces carry propagation delay

Faster arithmetic operations

Suitable for high-speed processors

Improves real-time performance

3. Booth's Algorithm for Signed Multiplication

Booth's algorithm is used for multiplication of signed binary numbers using 2's complement. The problem may occur because of incorrect bit-pair checking, arithmetic shifting, or sign extension.

Important Registers

Booth's algorithm uses:

$M \rightarrow$ Multiplicand

$Q \rightarrow$ Multiplier

$A \rightarrow$ Accumulator

$Q-1 \rightarrow$ Extra bit

Initially:

$$A = 0$$

$Q =$ Multiplier

$M =$ Multiplicand

$$Q-1 = 0$$

Bit-Pair Checking

Check ($Q_0, Q-1$):

After the operation, perform an arithmetic right shift on ($A, Q, Q-1$).

Possible Errors

Checking the wrong bit pair.

Performing logical shift instead of arithmetic shift.

Not preserving the sign bit.

Incorrect 2's complement subtraction.

Incorrect number of iterations.

Correction

For every iteration:

Check Q0 and Q-1

↓

Perform addition/subtraction

↓

Arithmetic right shift

↓

Repeat n times

The final product is obtained from:

(A,Q)

4. Restoring Division and Non-Restoring Division

The restoring division algorithm repeatedly subtracts the divisor from the partial remainder. If the result becomes negative, the divisor is added back. This restoration increases execution time.

Restoring Division Steps

Initialize the accumulator/remainder.

Shift the partial remainder and quotient.

Subtract the divisor.

Check the sign.

If negative, restore the previous remainder.

Set quotient bit.

Repeat for all bits.

The inefficient operation is:

Subtract divisor

↓

Result negative

↓

Add divisor again

This repeated restoration causes additional operations.

Optimized Solution: Non-Restoring Division

Non-restoring division avoids immediately restoring the remainder.

If the remainder is positive:

$$R = R - M$$

If the remainder is negative:

$$R = R + M$$

The decision depends on the sign of the previous remainder.

Advantages

Avoids repeated restoration

Requires fewer arithmetic operations

Faster execution

Better suitable for hardware implementation

Improves division performance

5. IEEE 754 Floating-Point Addition

IEEE 754 single precision uses 32 bits:

Sign | Exponent | Fraction

1 8 23

When adding very small and very large numbers, accuracy problems can occur because the exponents may be very different.

Floating-Point Addition Steps

1. Compare Exponents

The numbers must first have the same exponent.

2. Align Mantissas

The mantissa of the number with the smaller exponent is shifted right.

Example:

$$1.101 \times 2^5$$

$$0.001 \times 2^5$$

The smaller number may lose some significant bits during alignment.

3. Add Mantissas

After alignment:

Mantissa 1

+ Mantissa 2

Result

4. Normalize

The result is converted into normalized form.

For example:

$$10.101 \times 2^4$$

becomes:

$$1.0101 \times 2^5$$

5. Rounding

If more bits are present than can fit in the fraction field, rounding is performed.

Main Problem

When a very small number is added to a very large number, the small number may become too small to affect the final result after exponent alignment. This is called loss of significance or cancellation/precision loss depending on the operation.

Optimization Techniques

1. Use double precision

Double precision provides more bits for the significand and exponent, giving better accuracy.

2. Use suitable rounding

IEEE 754 rounding modes can reduce numerical errors.

3. Use guard and rounding bits

These extra bits help produce a more accurate rounded result.

4. Algorithmic adjustment

Numerical algorithms can be designed to reduce repeated rounding and precision loss.

Q0 Q-1 | Operation

00 | No operation

01 | $A = A + M$

10 | $A = A - M$

11 | No operation